

Neural Networks

Gunhee Kim



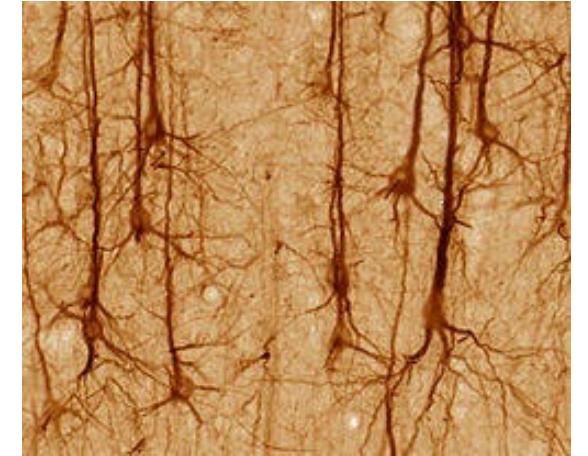
서울대학교
SEOUL NATIONAL UNIVERSITY

Outline

- Artificial Neurons
- Feedforward Neural Networks

Biological Analogy

- The brain is composed of a mass of interconnected neurons
 - Neurons transmit signals to each other
- What is the neuron?
 - The cell that performs information processing in the brain
 - Fundamental functional unit of all nervous system tissue
- Whether a signal is transmitted is an all-or-nothing event
 - The electrical potential in the cell of the neuron is thresholded
 - Whether a signal is sent, depends on the strength of the bond (synapse) between two neurons



Comparison of Brain and Computer

	Human	Computer
Processing Elements	100 Billion neurons	10 Million gates
Interconnects	1000 per neuron	A few
Cycles per sec	1000	500 Million
Bandwidth	10 Trillion bits/sec	1 Billion bits/sec
2X improvement	200,000 Years	2 Years

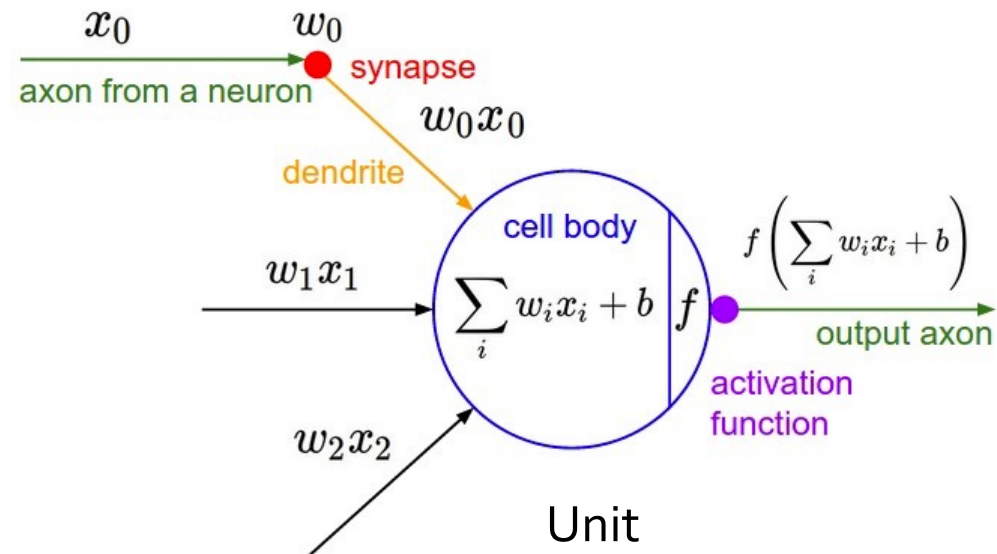
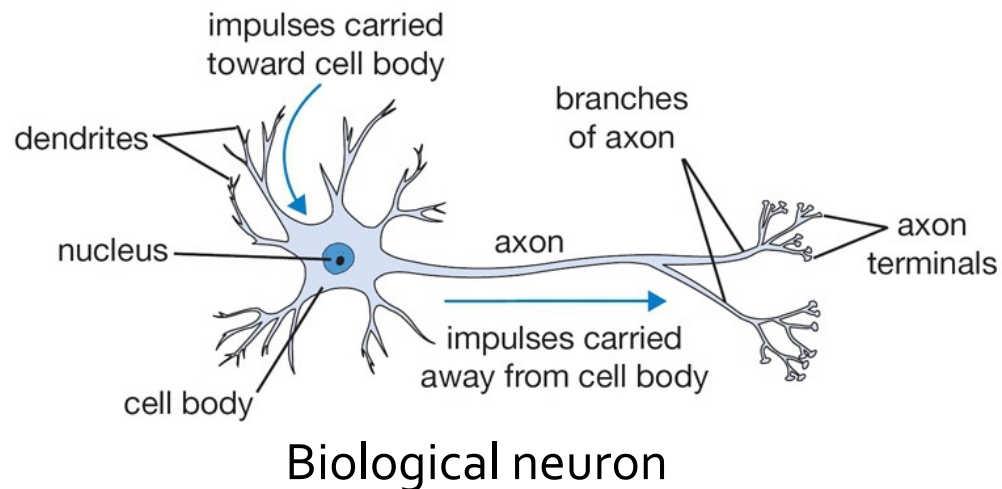
- Neurons are slow!
 - Neurons are a million times slower than gates
- Parallelism & Connectivity
 - The brain can fire all the neurons in a single step
 - Connected via approximately 10^{15} synapses

What is Neural Network?

- Artificial Neural Networks (ANN) or Multi-Layer Perceptrons (MLP)
 - An artificial neural network is a system composed of many simple processing elements operating in parallel that can acquire, store, and utilize experiential knowledge
- A biological neuron is not like an artificial neuron
 - Perform complex nonlinear computations
 - Complex non-linear dynamical system (timing is important)

Neurons vs Units

- Each element of ANN is a node called **unit**
 - Units are connected by links
 - Each link has a numeric weight
 - Real neuron is far away from our simplified model - unit

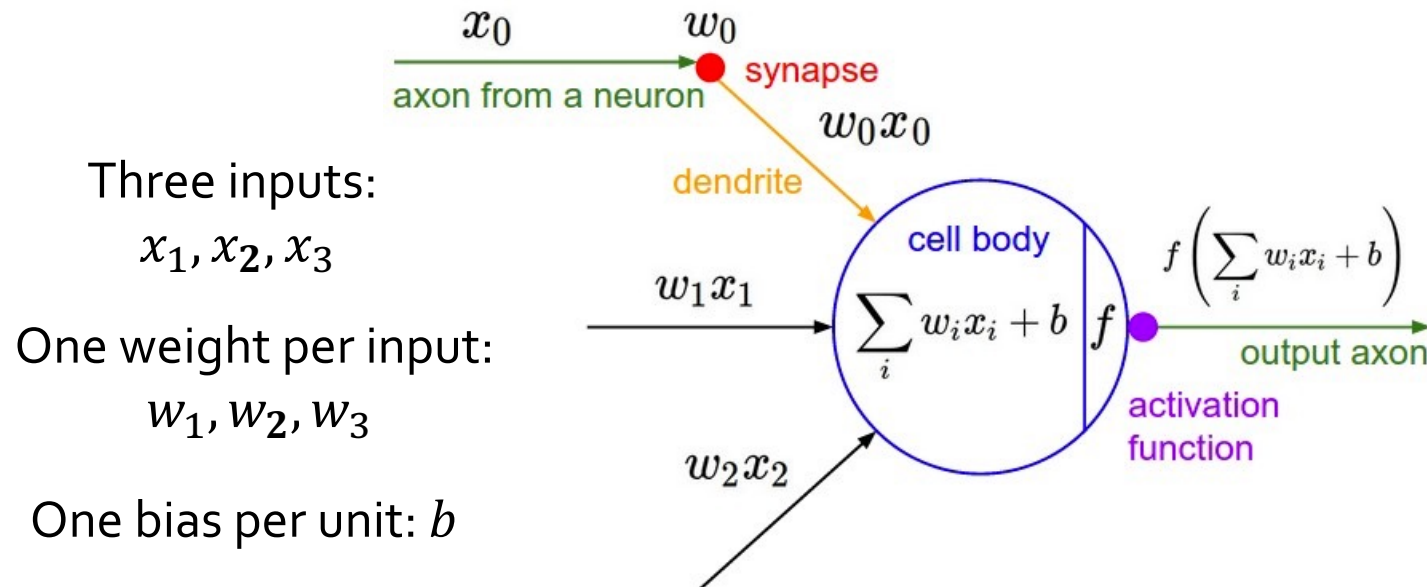


Simple Computations in Units

- Linear input function
 - Calculate weighted sum of all inputs
- Nonlinear activate function
 - Transform sum into activation level as a single output

$$h = \sum_i w_i x_i + b$$

Weights Bias



$$f(h) = f\left(\sum_i w_i x_i + b\right)$$

$$f(h) = f(w_1 x_1 + w_2 x_2 + w_3 x_3 + b)$$

Activation Functions

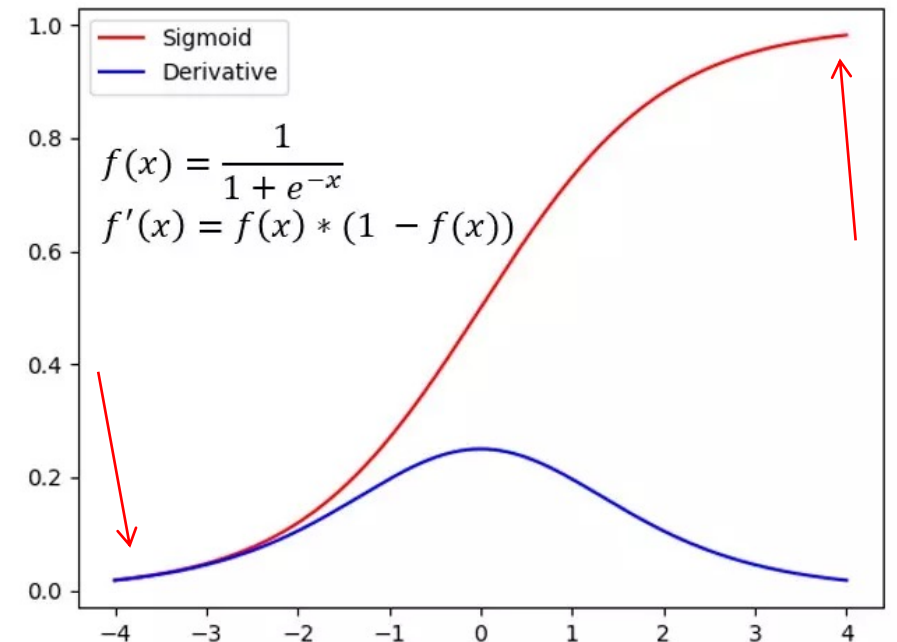
- 1. Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- Squashes numbers to range [0,1]
- Historically popular since they have nice interpretation as a saturating firing rate of a biological neuron

- Two major issues

- 1. Saturated neurons kill the gradient
- 2. Sigmoid outputs are not zero-centered (always positive)
- $\mathbf{w}$ is always all positive or all negative (zig-zagging dynamics in the gradient update)

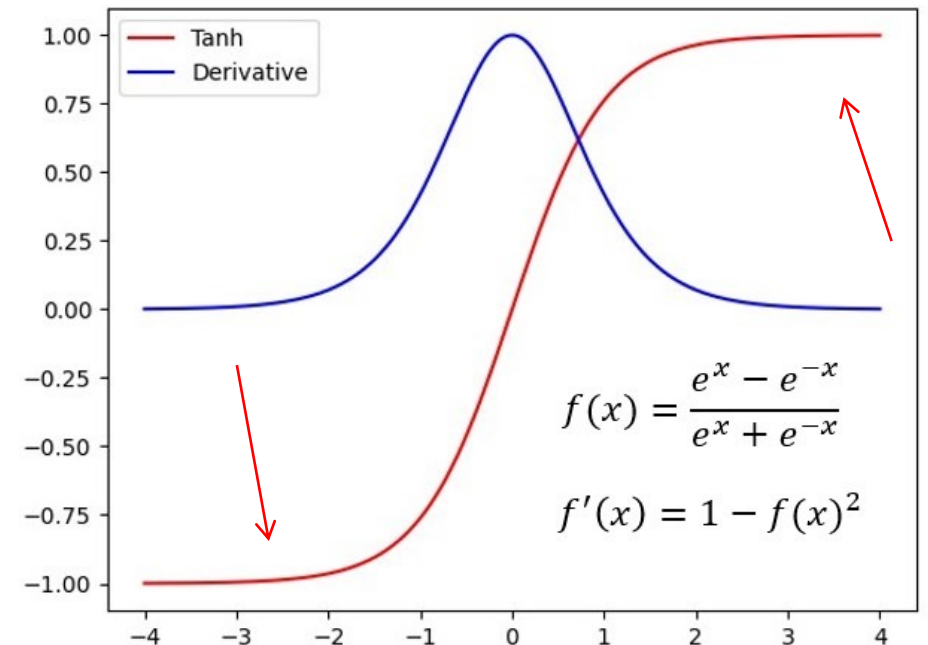


Activation Functions

- 2. Tanh (hyperbolic tangent function)

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- Squashes numbers to range $[-1,1]$
- Zero-centered (better than Sigmoid)
- The issue still holds
 - 1. Saturated neurons kill the gradient



Activation Functions

- 3. ReLU (Rectified Linear Unit)

$$f(x) = \max(0, x)$$

- Does not saturate and very computationally efficient
- Converges much faster than sigmoid/tanh in practice

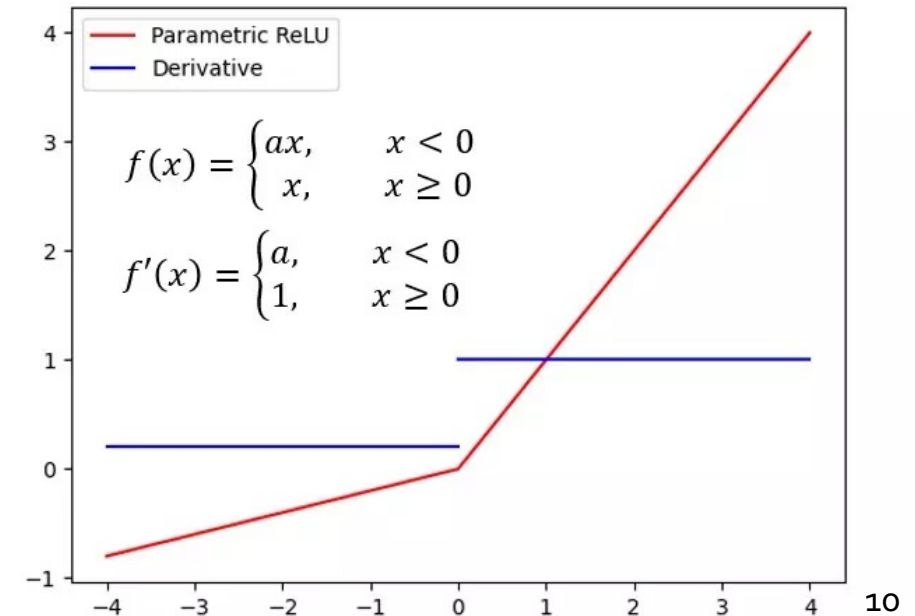
- One issue is ... some units are never activated

- i.e., gradient is 0 when $x < 0$

- 4. Parametric ReLU

$$f(x) = 1(x < 0)ax + 1(x \geq 0)x \text{ with a small } a$$

- At $a = 0$, it becomes ReLU: $f(x) = \max(0, x)$



Practical Remarks on Activation (in CNN)

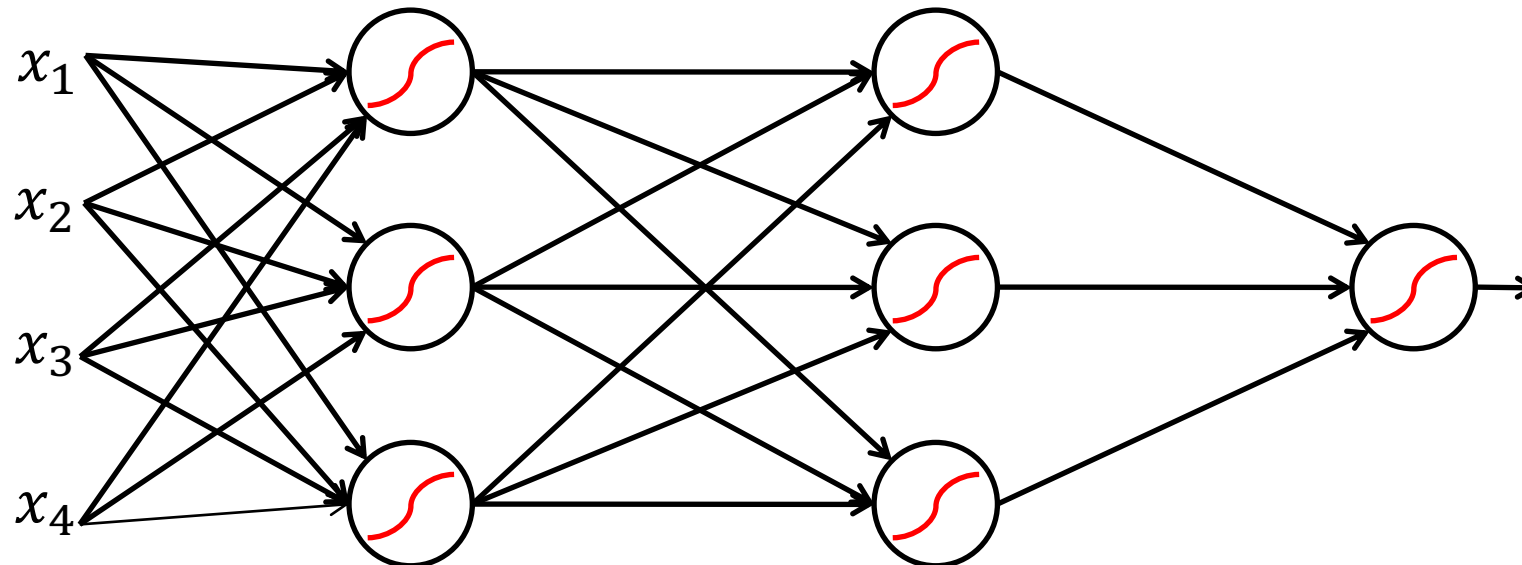
- Use ReLU!
 - ReLU is a good default choice for most problems
 - Be careful with learning rates and monitor the fraction of *dead* units in a network
 - Could try out Leaky ReLU or tanh
 - Do not use sigmoid anymore
- Stay tuned! The above arguments can be nullified!
 - Recently popular ones: Exponential linear units (ELU), Gaussian error linear unit (GELU), Swish function, ...
 - A survey paper: V. Kunc & J. Kléma, Three Decades of Activations: A comprehensive Survey of 400 Activation Functions for Neural Networks, arXiv 2024

Outline

- Artificial Neurons
- Feedforward Neural Networks

Feedforward Neural Networks

- Flexible architecture
 - Can be arranged in any multiple layers
 - Can have any multiple input/output units
- Each unit is linked only with the units in next layer
 - No unit is linked btw. the same layer, back to the previous layer or skipping a layer



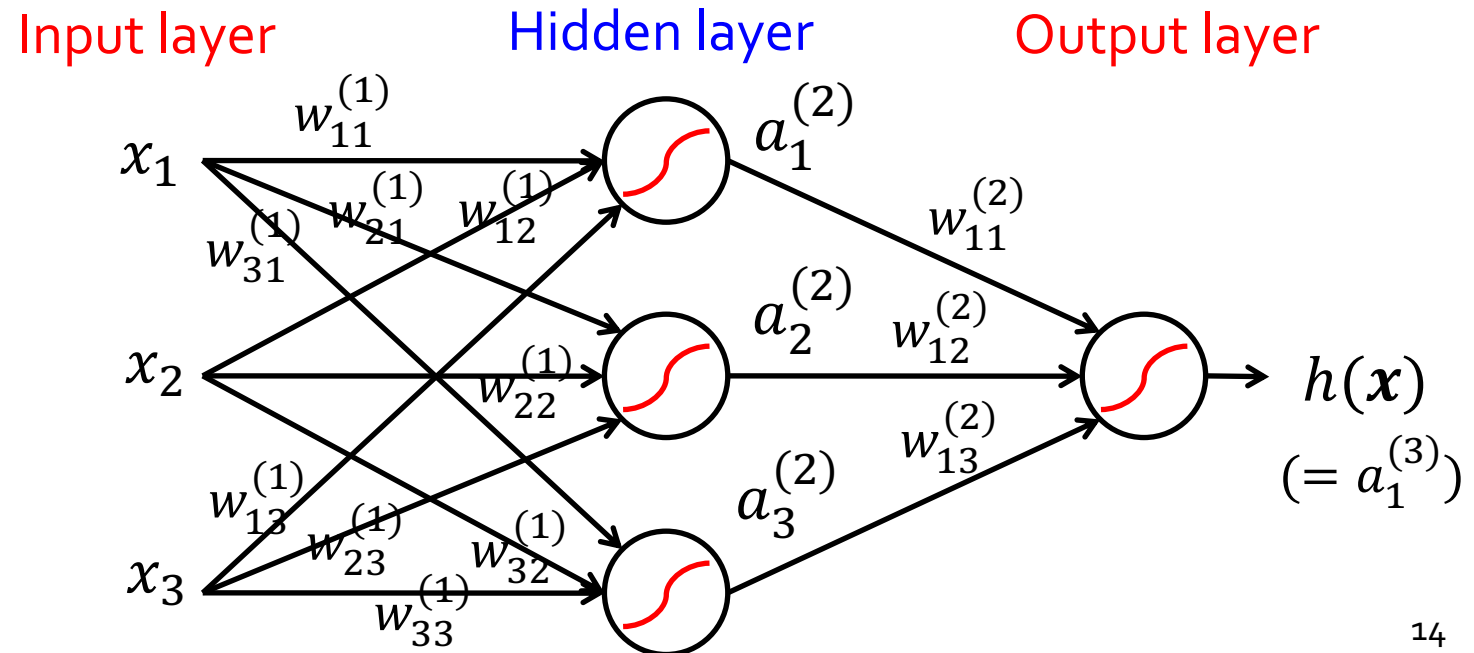
Feedforward Neural Networks

- The notation

- $w_{ij}^{(l)}$: the weight from unit j of layer l to unit i of layer $l + 1$
- $a_i^{(l)}$: activation (i.e., output value) of unit i in layer l
- $a_i^{(1)} = x_i$: i -th input
- L : # layers

- # parameters = # W + # b

- # W = # edges
- # b = # nodes



Forward Propagation

$$a_1^{(2)} = f\left(W_{11}^{(1)}x_1 + W_{12}^{(1)}x_2 + W_{13}^{(1)}x_3 + b_1^{(1)}\right) = f(z_1^{(2)})$$

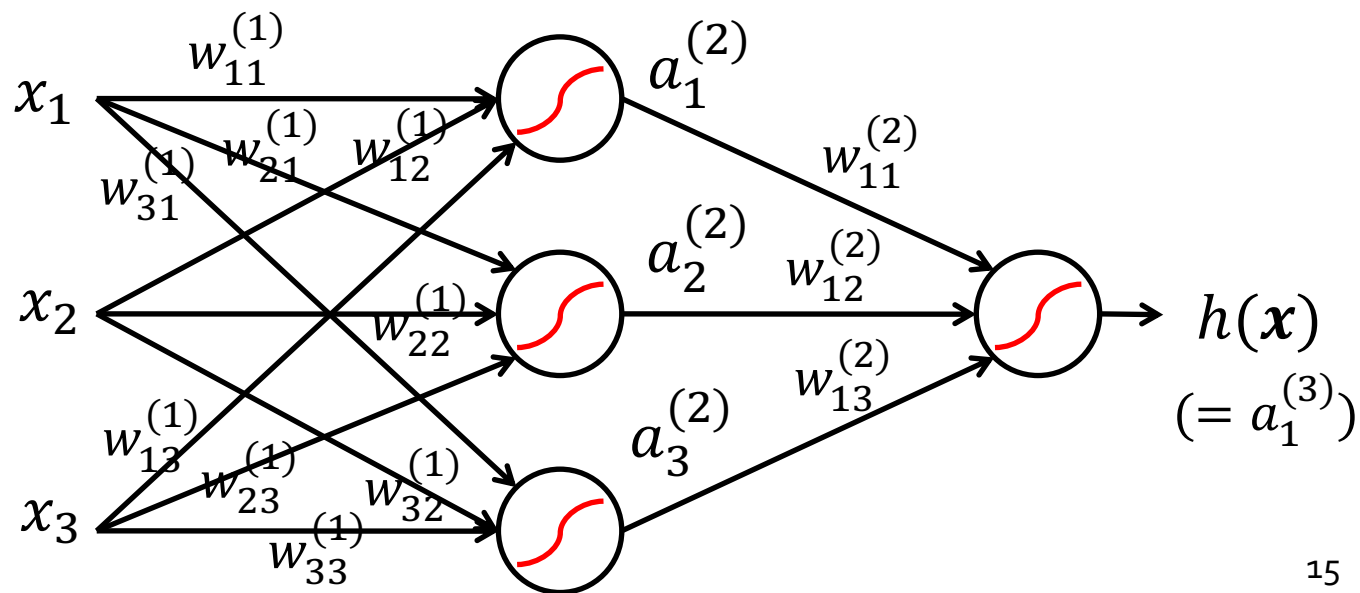
$$a_2^{(2)} = f(W_{21}^{(1)}x_1 + W_{22}^{(1)}x_2 + W_{23}^{(1)}x_3 + b_2^{(1)}) = f(z_2^{(2)})$$

$$a_3^{(2)} = f(W_{31}^{(1)}x_1 + W_{32}^{(1)}x_2 + W_{33}^{(1)}x_3 + b_3^{(1)}) = f(z_3^{(2)})$$

$$h(x) = a_1^{(3)} = f\left(W_{11}^{(2)}a_1^{(2)} + W_{12}^{(2)}a_2^{(2)} + W_{13}^{(2)}a_3^{(2)} + b_1^{(2)}\right) = f(z_1^{(3)})$$

$z_i^{(l)}$: the total weighted sum of inputs to unit i in layer l

e.g., $z_i^{(2)} = \sum_{j=1}^n W_{ij}^{(1)}x_j^{(1)} + b_i^{(1)}$



Forward Propagation in Matrix Form

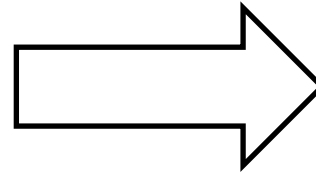
$$\mathbf{z}^{(2)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}$$

$$\mathbf{a}^{(2)} = f(\mathbf{z}^{(2)})$$

$$\mathbf{z}^{(3)} = \mathbf{W}^{(2)}\mathbf{a}^{(2)} + \mathbf{b}^{(2)}$$

$$h(\mathbf{x}) = \mathbf{a}^{(3)} = f(\mathbf{z}^{(3)})$$

We recursively
compute



$$\mathbf{a}^{(1)} = \mathbf{x}$$

$$\mathbf{z}^{(l+1)} = \mathbf{W}^{(l)}\mathbf{a}^{(l)} + \mathbf{b}^{(l)}$$

$$\mathbf{a}^{(l+1)} = f(\mathbf{z}^{(l+1)})$$

